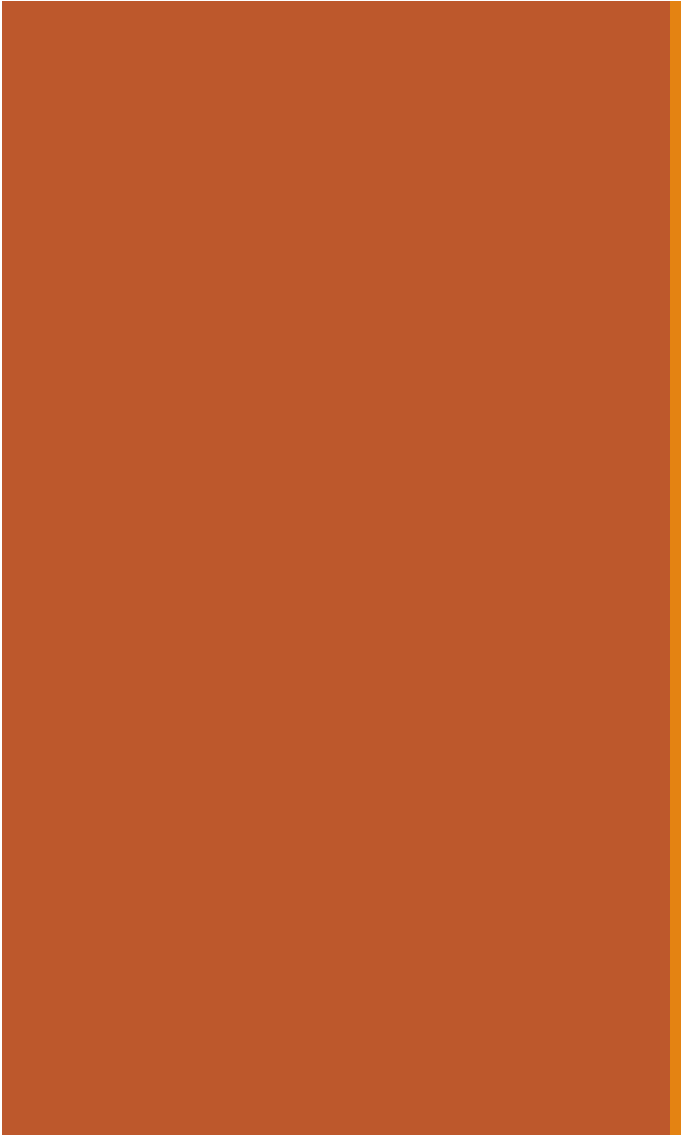


26: Intro to Deep Learning

Jerry Cain
May 31, 2024

[Lecture Discussion on Ed](#)



Deep Learning

Innovations in deep learning



AlphaGO (2016)

Lisa Yan, Chris Piech, Mehran Sahami, and Jerry Cain, CS109, Spring 2024

Making history

AlphaGo is the first computer program to defeat a professional human Go player, the first to defeat a Go world champion, and is arguably the strongest Go player in history.

Deep learning and neural networks are core theories and technologies behind the current AI revolution.

Errata:

- Checkers is the last **solved** game (from game theory, where perfect player outcomes can be fully predicted from any gameboard).
https://en.wikipedia.org/wiki/Solved_game
- The first machine learning algorithm defeated a world champion in Chess in 1996.
[https://en.wikipedia.org/wiki/Deep_Blue_\(chess_computer\)](https://en.wikipedia.org/wiki/Deep_Blue_(chess_computer))

Stanford University

Computers making art



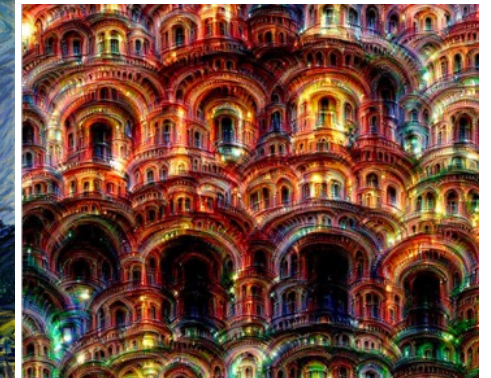
The Next Rembrandt

<https://medium.com/@DutchDigital/the-next-rembrandt-bringing-the-old-master-back-to-life-35dfb1653597>



A Neural Algorithm of Artistic Style

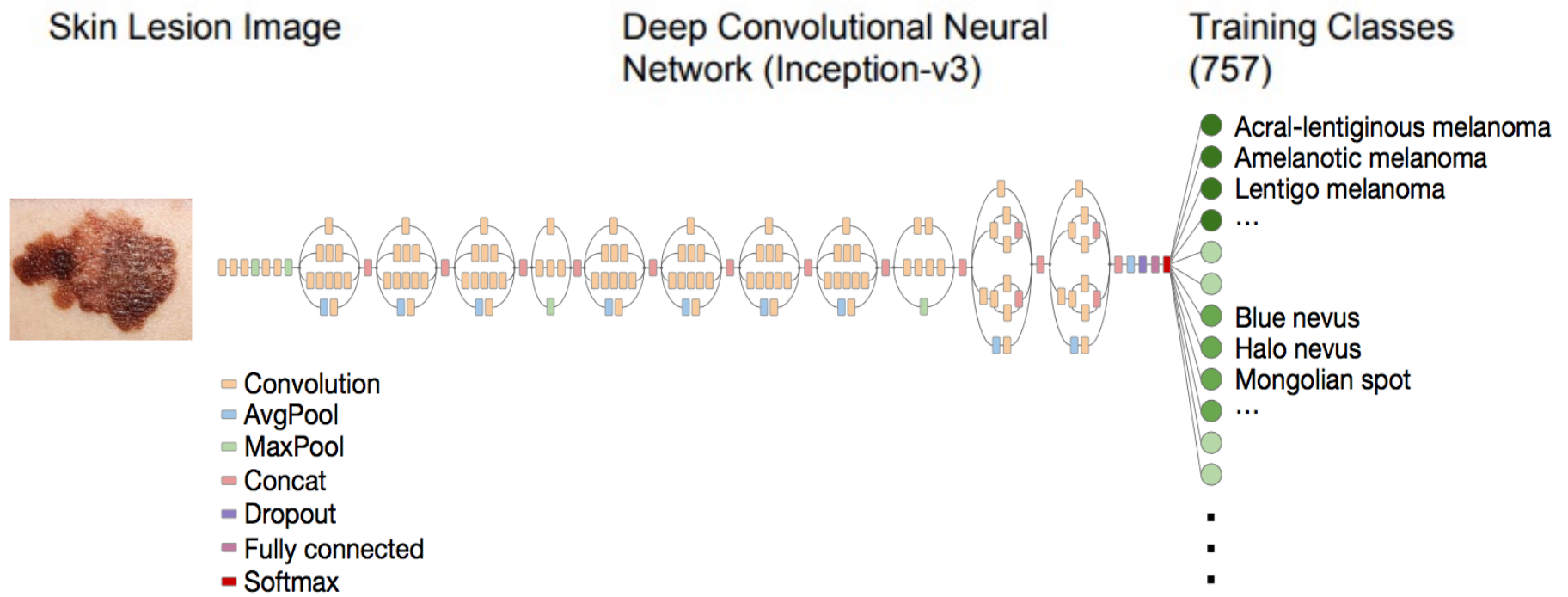
<https://arxiv.org/abs/1508.06576>
<https://github.com/jcjohnson/neural-style>



Google Deep Dream

<https://ai.googleblog.com/2015/06/inceptionism-going-deeper-into-neural.html>

Detecting skin cancer



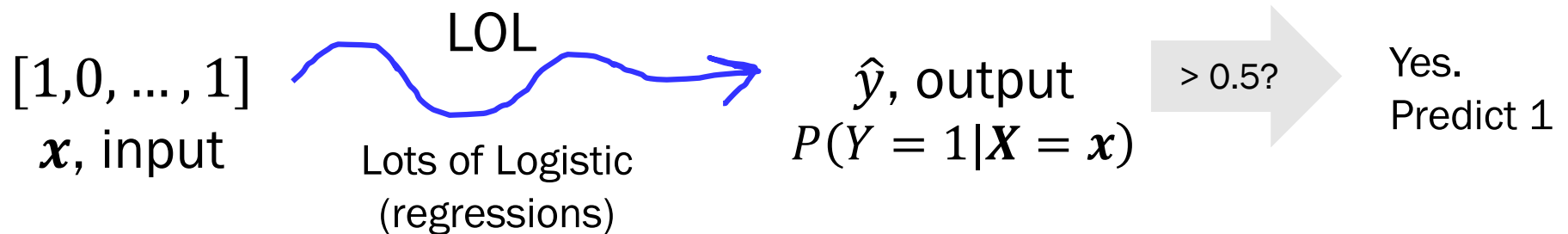
Esteva, Andre, et al. "Dermatologist-level classification of skin cancer with deep neural networks." *Nature* 542.7639 (2017): 115-118.

Lisa Yan, Chris Piech, Mehran Sahami, and Jerry Cain, CS109, Spring 2024

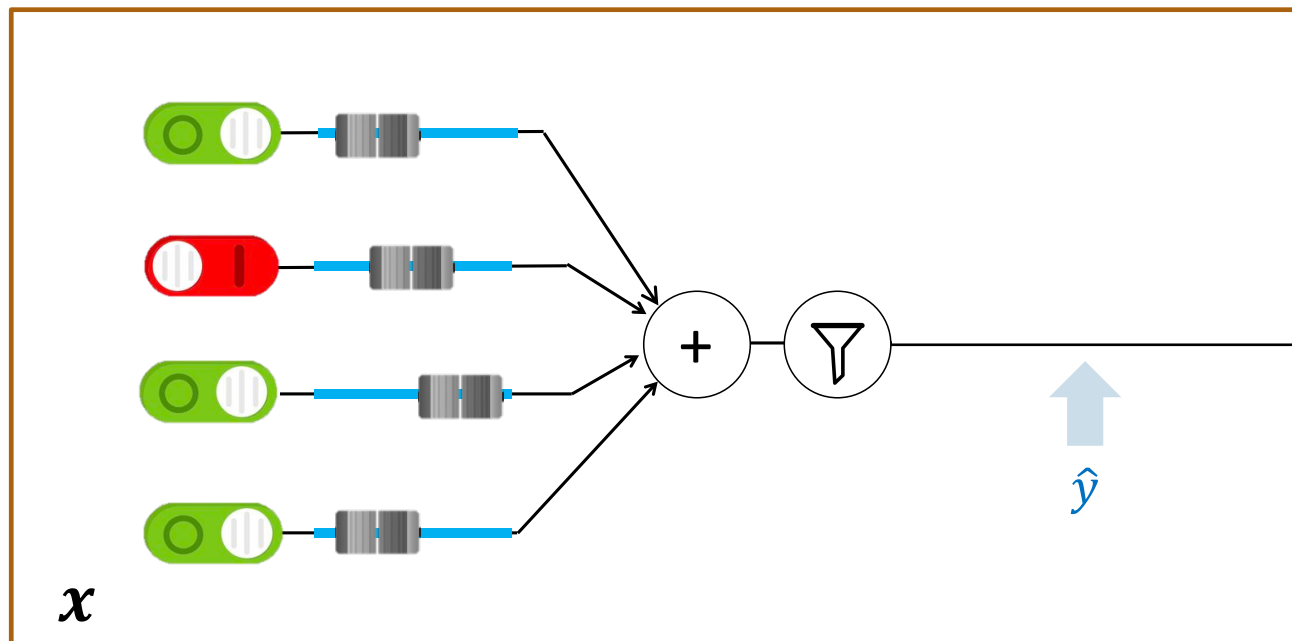
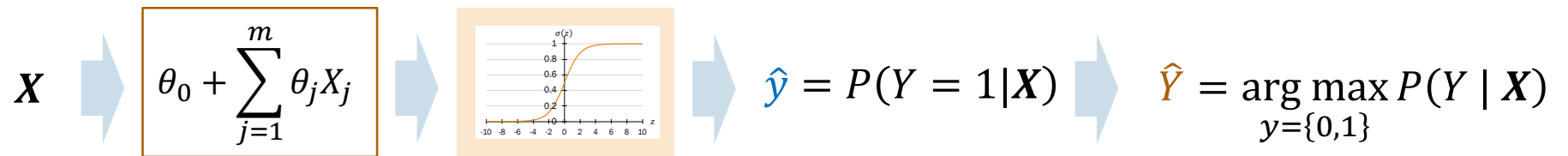
Deep learning

def **Deep learning** is
maximum likelihood estimation
with neural networks.

def A **neural network** is,
at its core, many logistic
regression units stacked on top
of each other and daisy-
chained together.



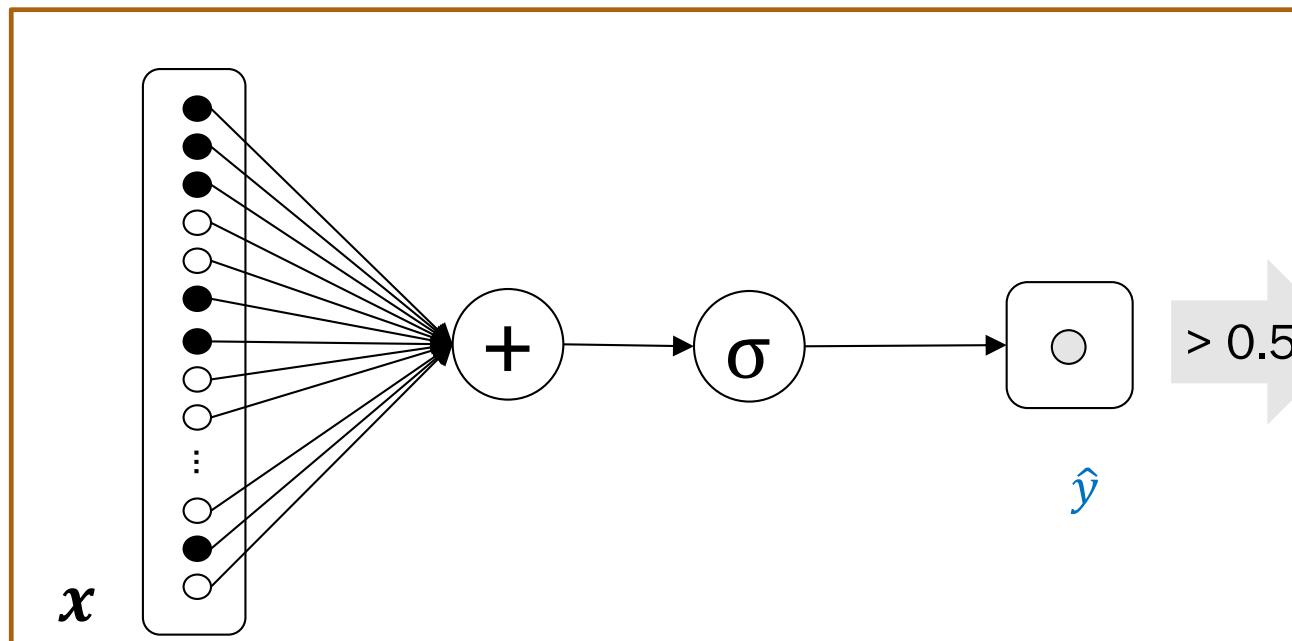
Logistic Regression Model



Let's focus on the model up to $\hat{y}$.

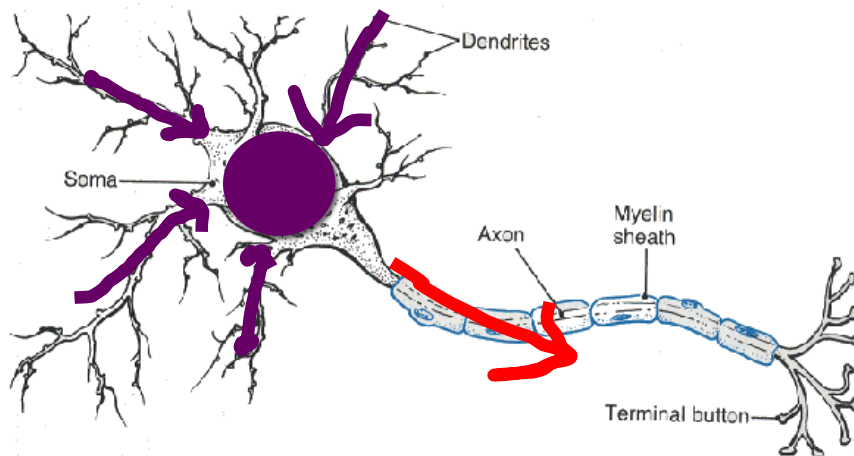
Logistic Regression Model

$$\mathbf{X} \Rightarrow \theta_0 + \sum_{j=1}^m \theta_j X_j \Rightarrow \sigma(z) \Rightarrow \hat{y} = P(Y = 1 | \mathbf{X}) \Rightarrow \hat{Y} = \arg \max_{y \in \{0,1\}} P(Y | \mathbf{X})$$

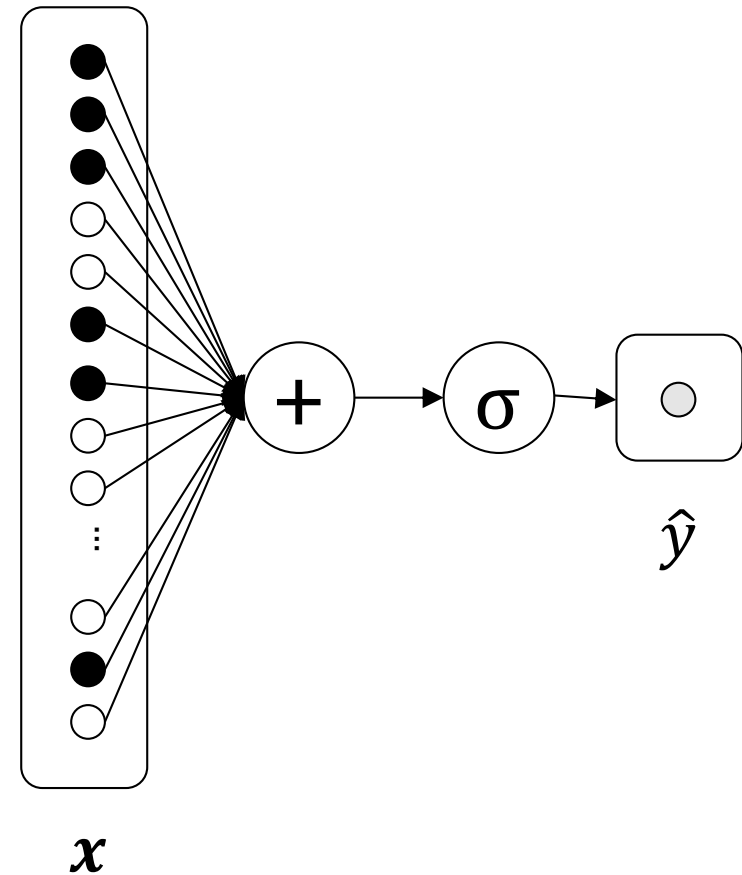


Let's focus on the model up to $\hat{y}$.

One neuron = One logistic regression

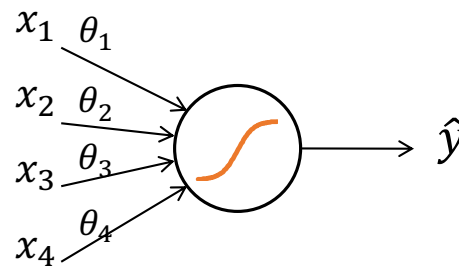
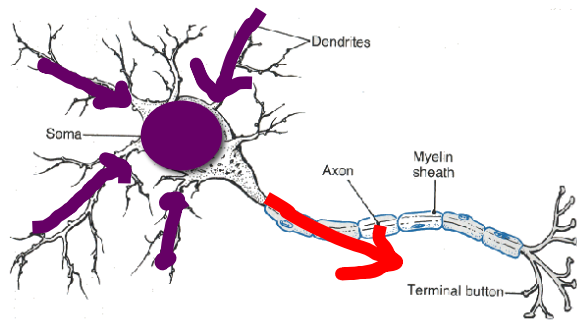


=



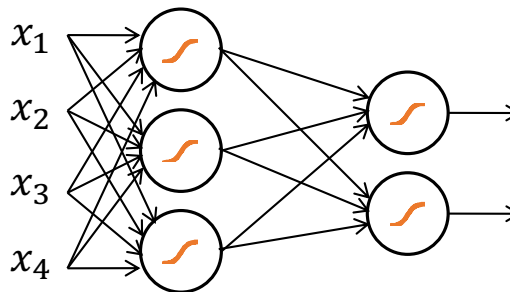
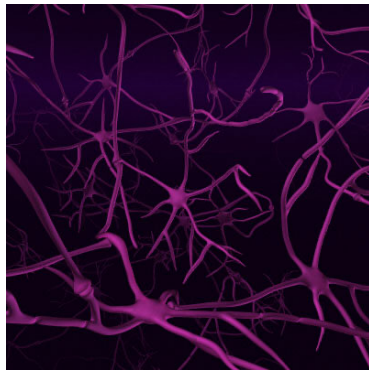
Biological basis for neural networks

A neuron



One neuron =
one logistic
regression

Your brain

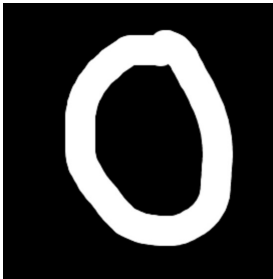


Neural network =
many logistic
regressions

(or rather, someone else's brain)

Digit recognition example

Input image



Input feature vector

$$\mathbf{x}^{(i)} = [0,0,0,0, \dots, 1,0,0,1, \dots, 0,0,1,0]$$

Output label

$$y^{(i)} = 0$$

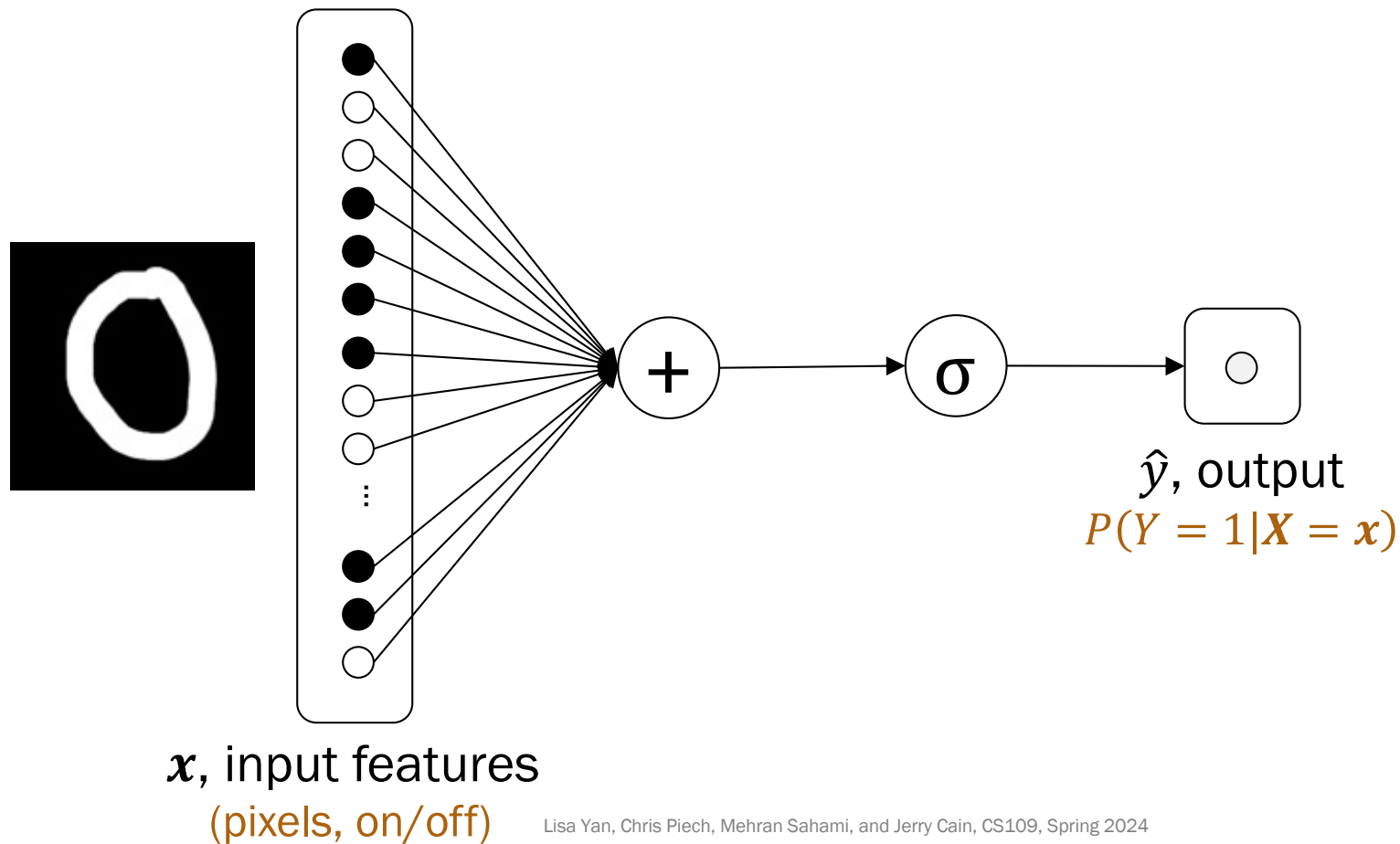


$$\mathbf{x}^{(i)} = [0,0,1,1, \dots, 0,1,1,0, \dots, 0,1,0,0]$$

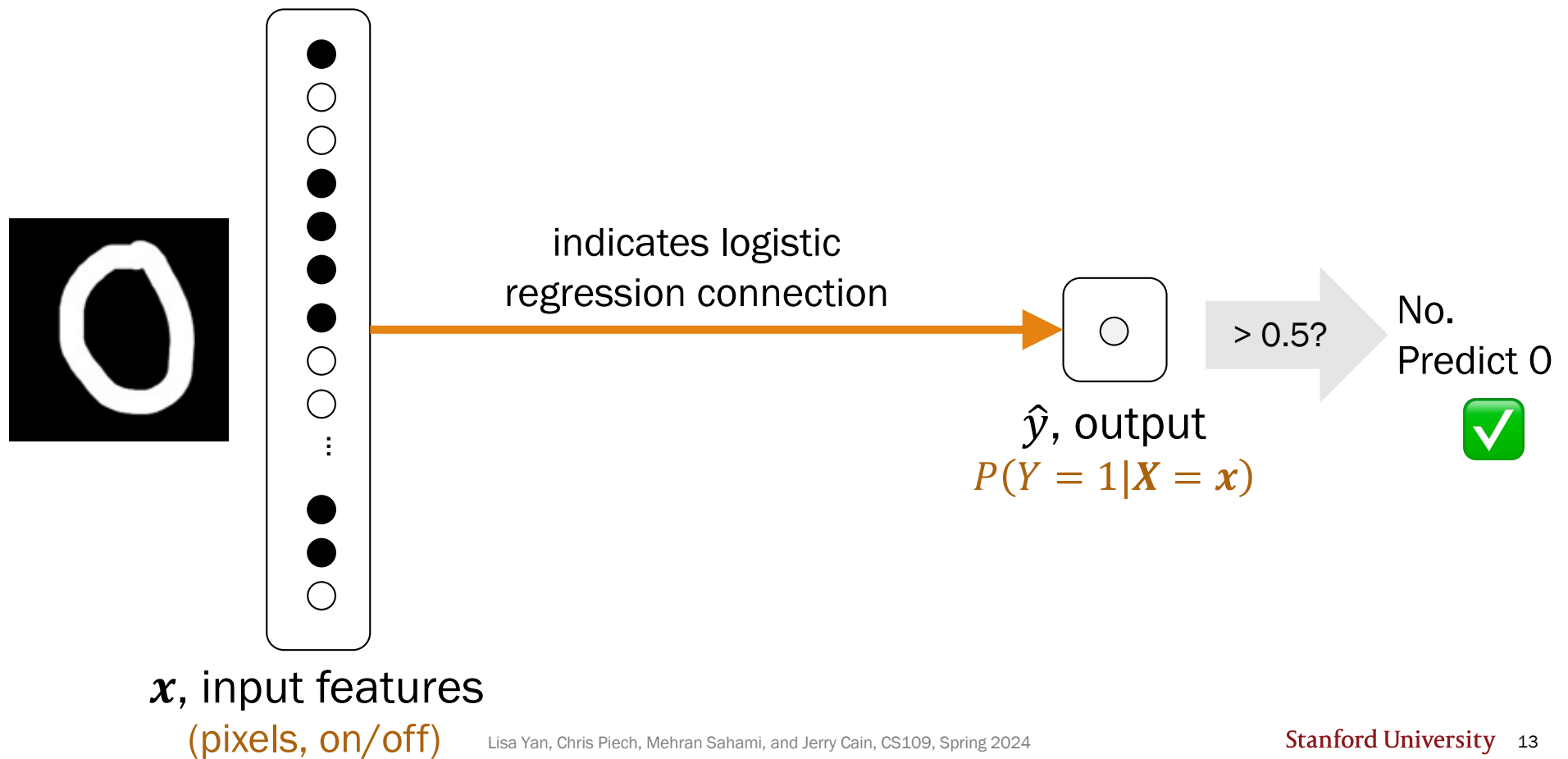
$$y^{(i)} = 1$$

We make feature vectors from digitized pictures of numbers.

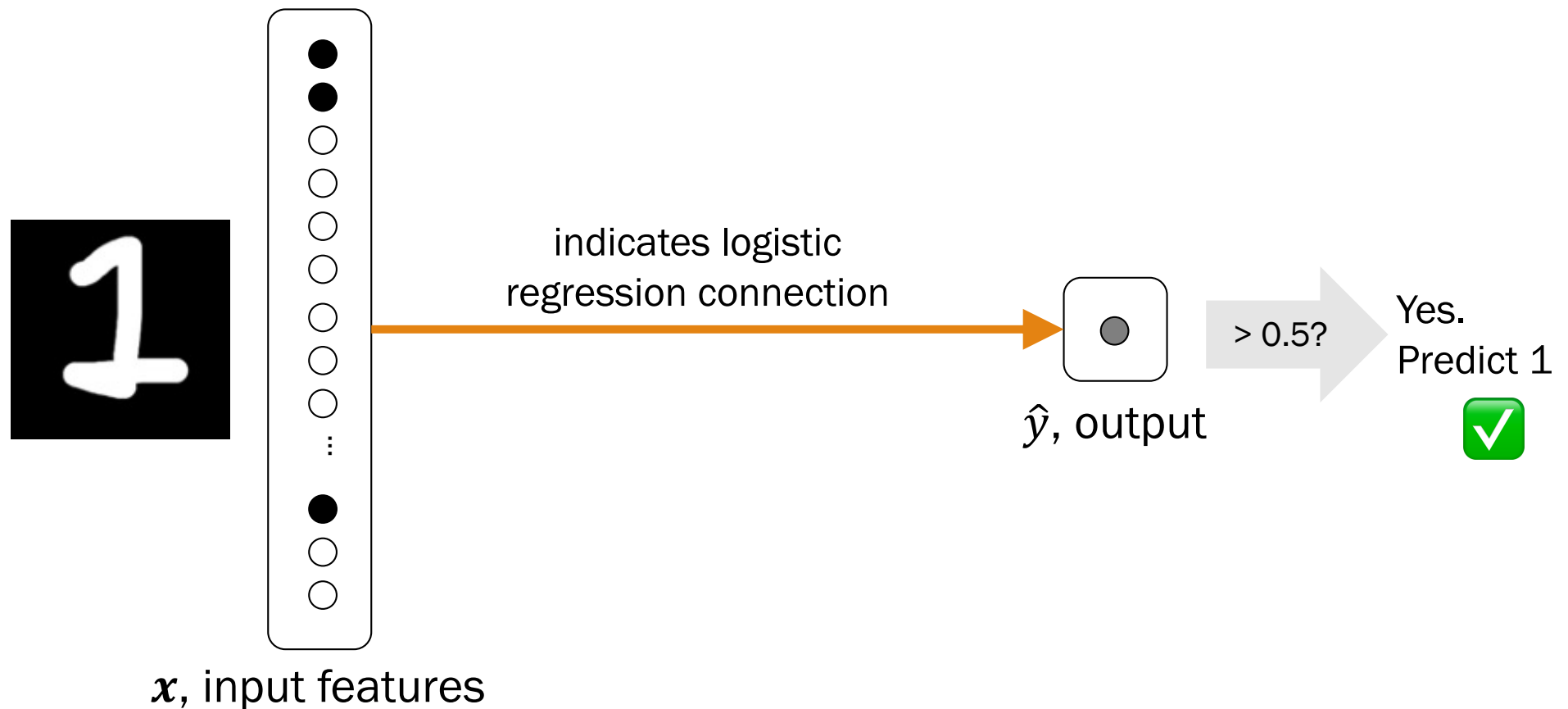
Logistic Regression



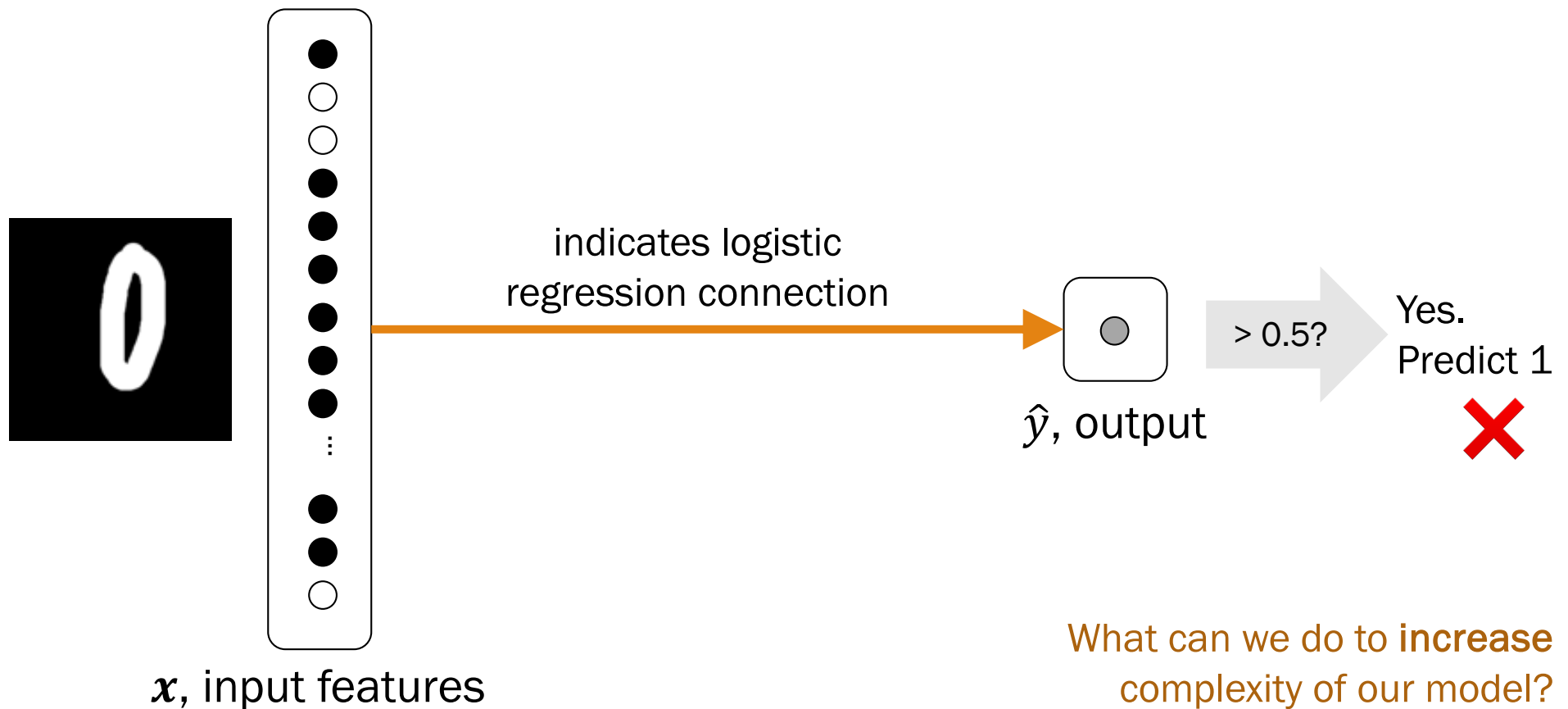
Logistic Regression



Logistic Regression



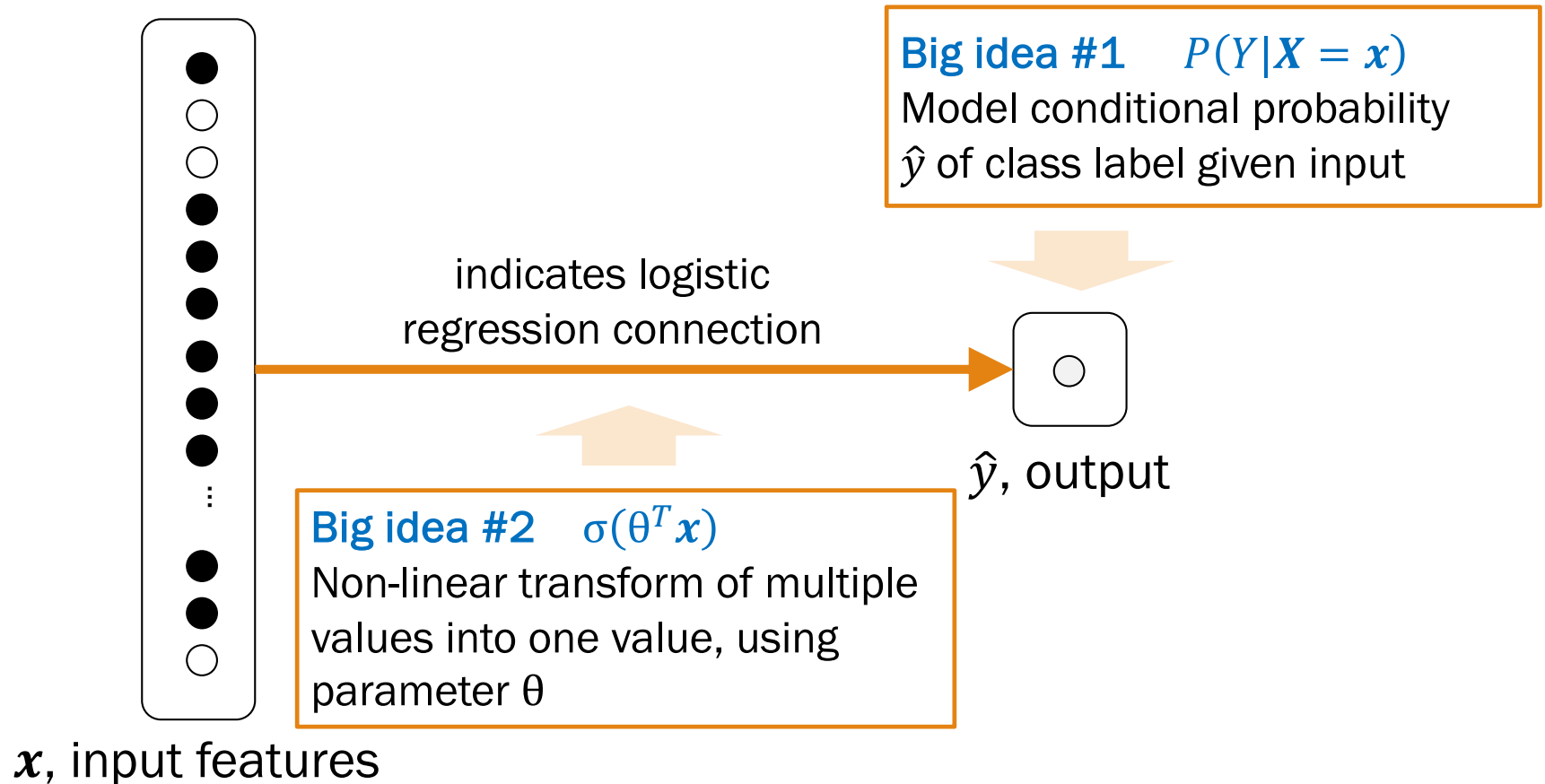
Logistic Regression



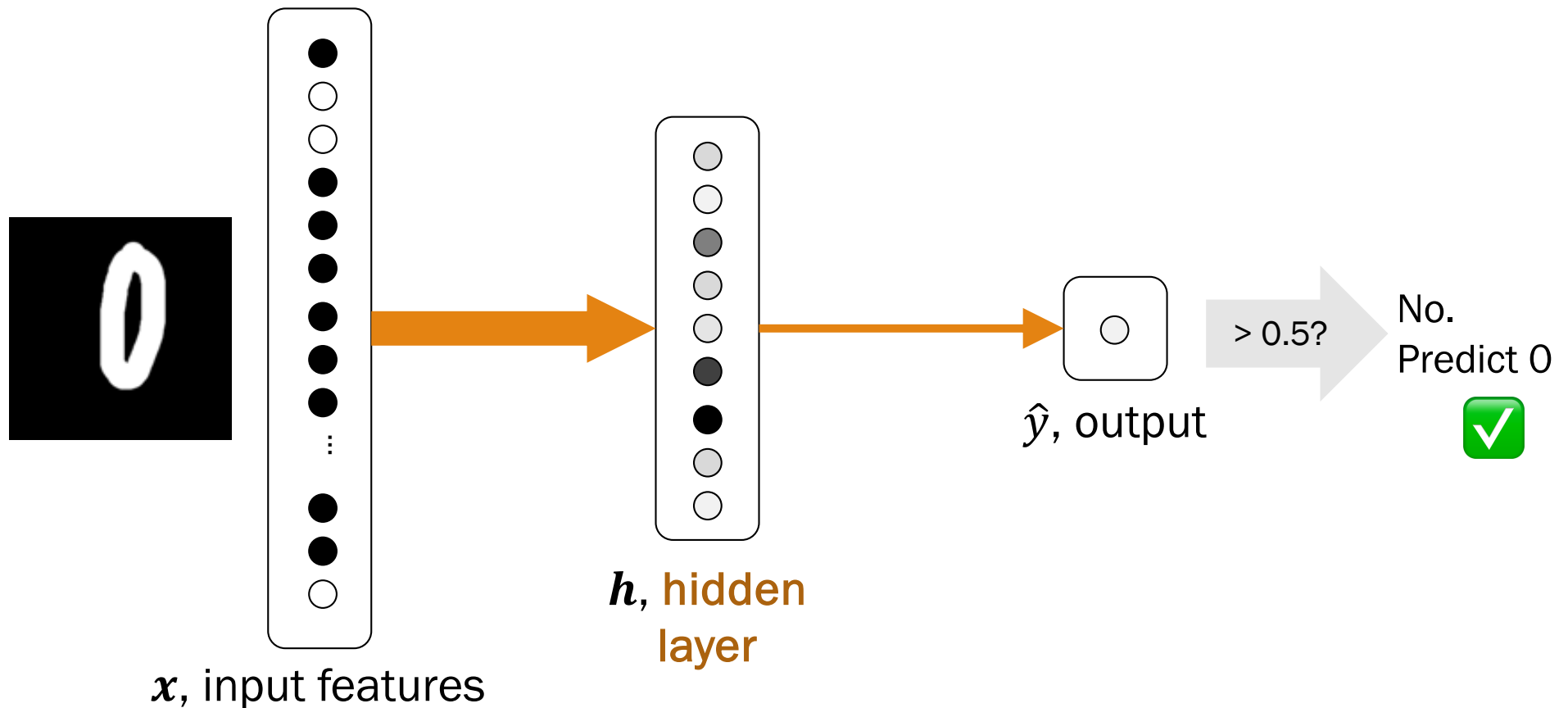
What can we do to **increase complexity** of our model?
i.e. sophistication

Take two big ideas from Logistic Regression

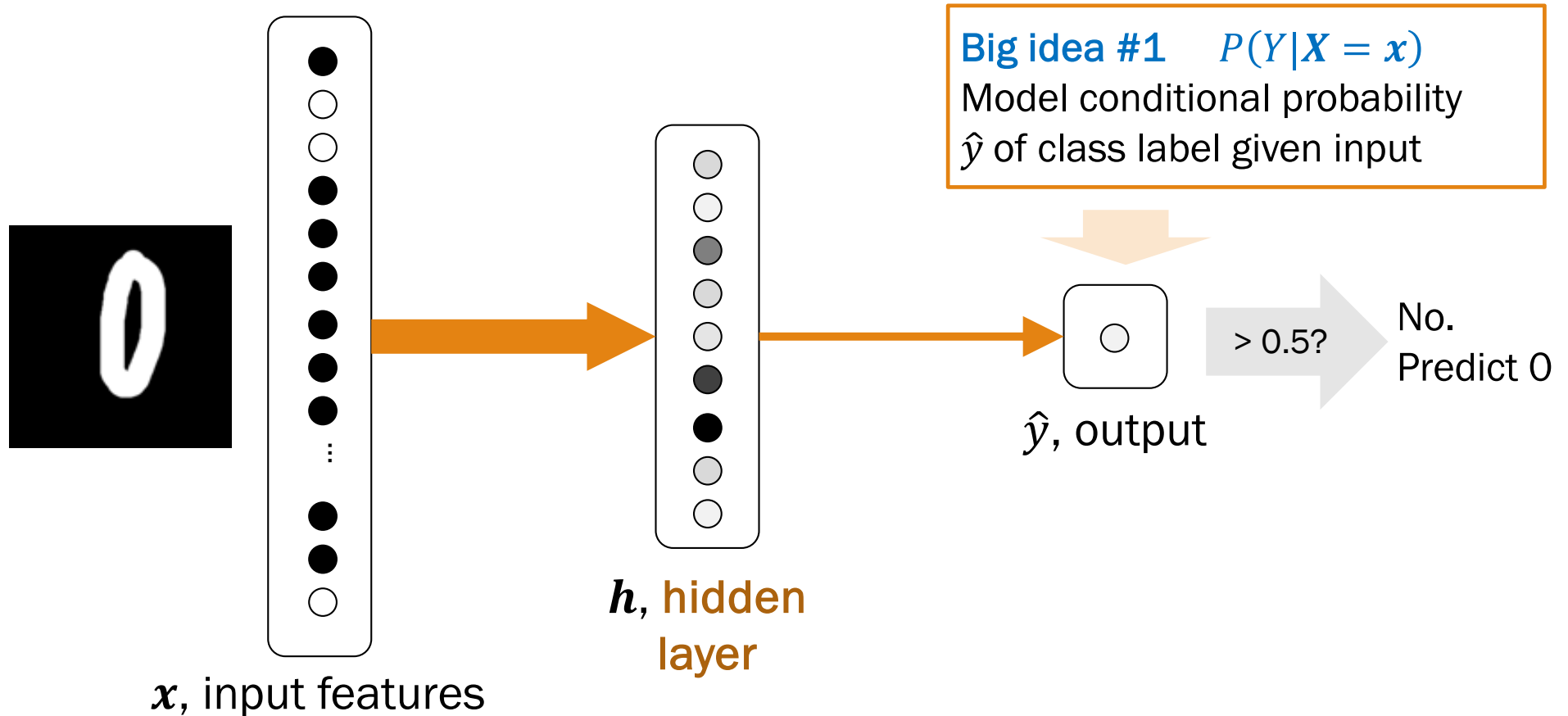
Review



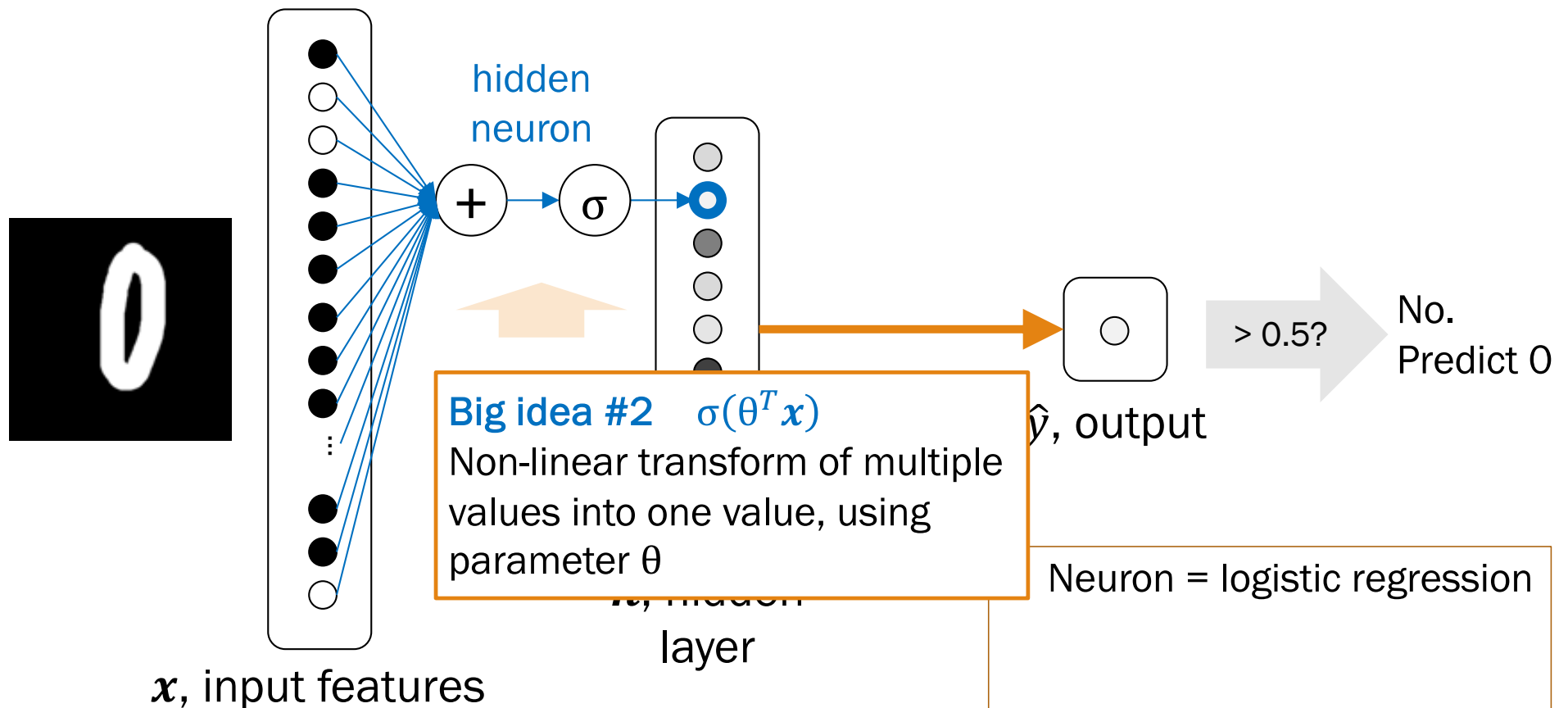
Introducing: The Neural network



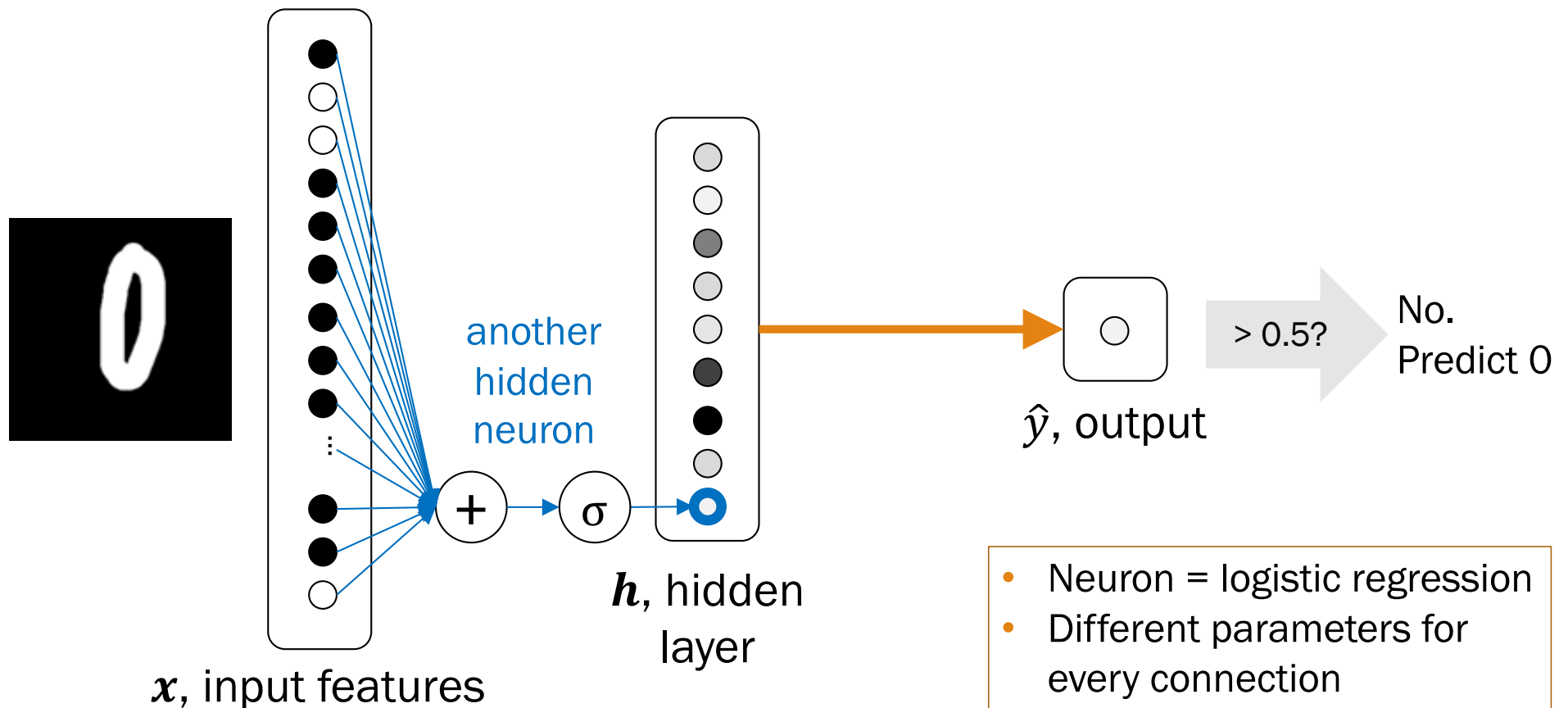
Neural network



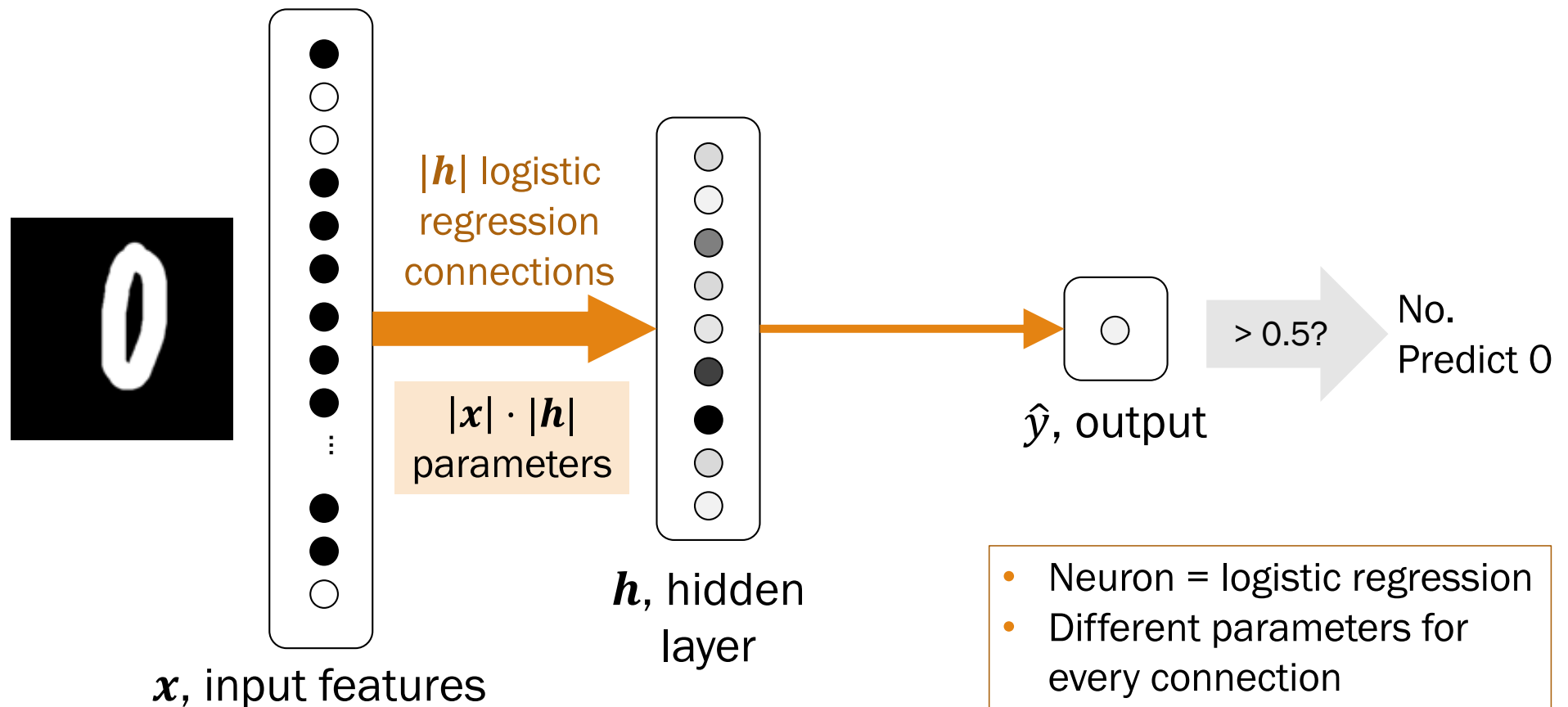
Feed neurons into other neurons



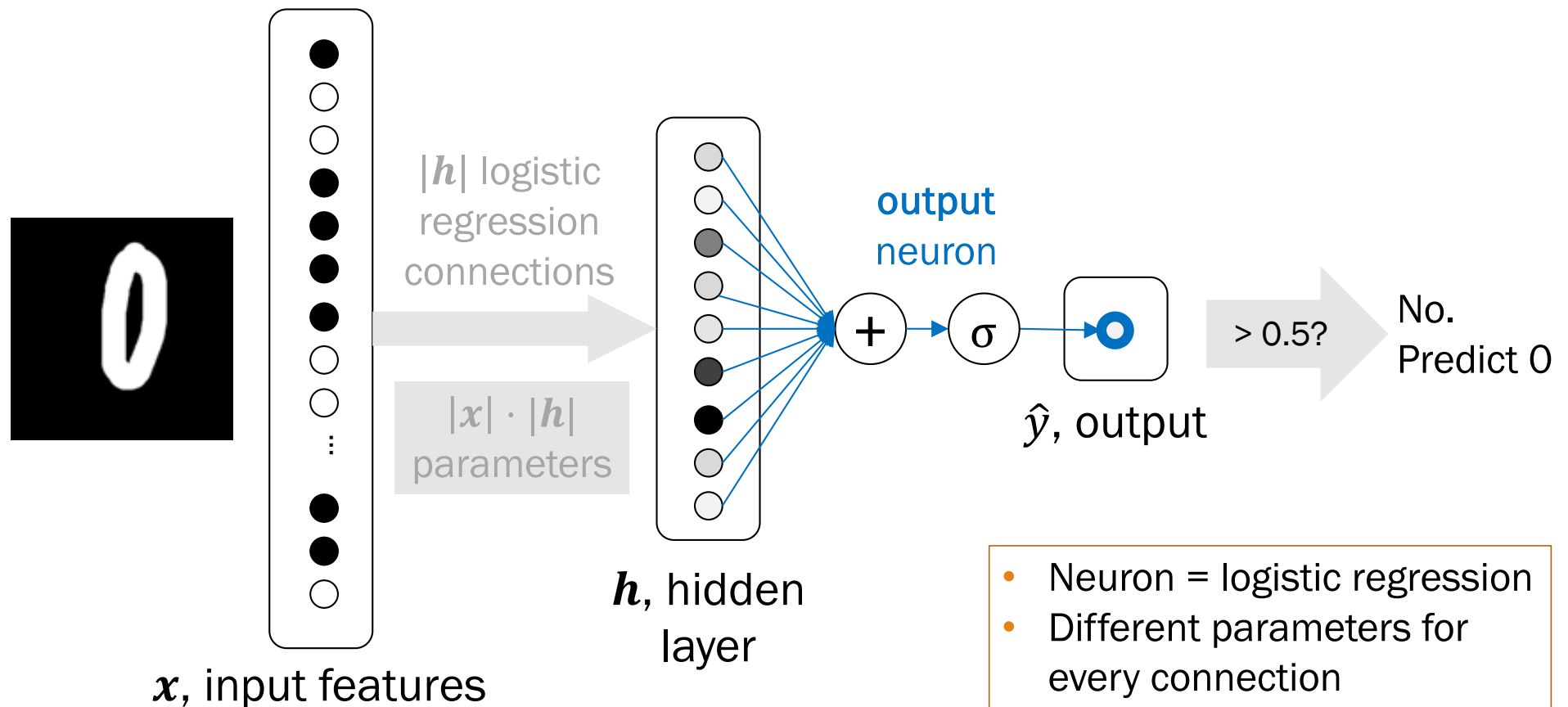
Feed neurons into other neurons



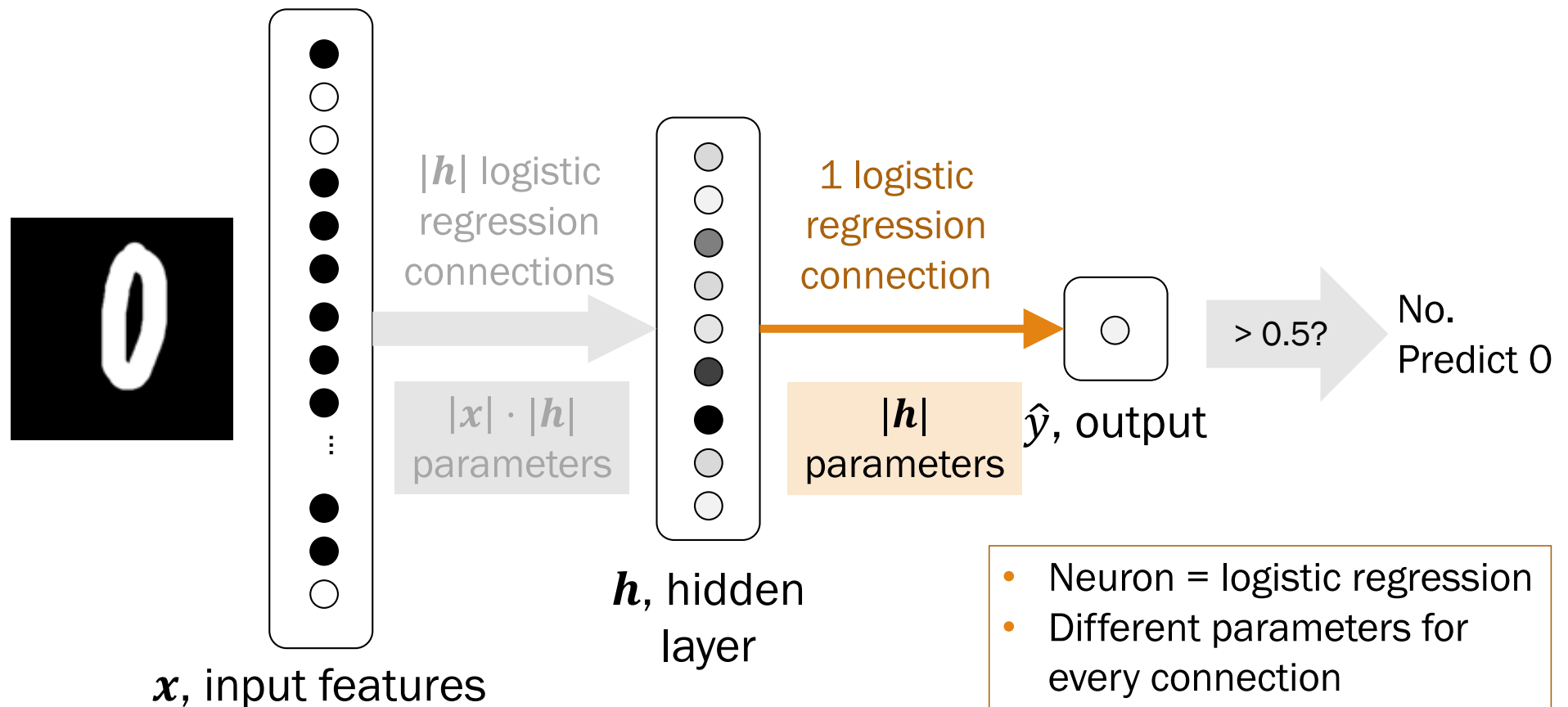
Feed neurons into other neurons



Feed neurons into other neurons



Feed neurons into other neurons



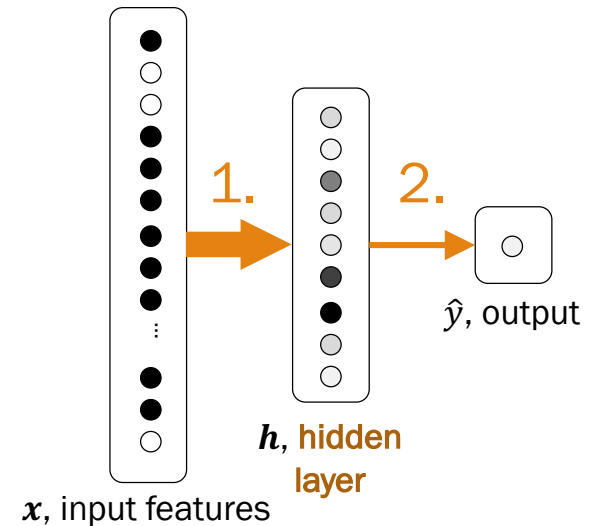
Why doesn't a linear model introduce “complexity”?

Neural network:

1. for $j = 1, \dots, |\mathbf{h}|$:

$$h_j = \sigma(\theta_j^{(h)T} \mathbf{x}) \quad \left. \vphantom{h_j = \sigma(\theta_j^{(h)T} \mathbf{x})} \right\} \text{sigmoid is present}$$

2. $\hat{y} = \sigma(\theta^{(\hat{y})T} \mathbf{h}) = P(Y = 1 | \mathbf{X} = \mathbf{x})$



Linear network:

1. for $j = 1, \dots, |\mathbf{h}|$:

$$h_j = \theta_j^{(h)T} \mathbf{x} \quad \left. \vphantom{h_j = \theta_j^{(h)T} \mathbf{x}} \right\} \text{sigmoid is absent}$$

2. $\hat{y} = \sigma(\theta^{(\hat{y})T} \mathbf{h}) = P(Y = 1 | \mathbf{X} = \mathbf{x})$



Why doesn't a linear model introduce “complexity”?

Neural network:

1. for $j = 1, \dots, |\mathbf{h}|$:

$$h_j = \sigma(\theta_j^{(h)^T} \mathbf{x})$$

2. $\hat{y} = \sigma(\theta^{(\hat{y})^T} \mathbf{h}) = P(Y = 1 | \mathbf{X} = \mathbf{x})$

assume $|\mathbf{x}| = |\mathbf{h}| = 2$

Linear network:

1. for $j = 1, \dots, |\mathbf{h}|$:

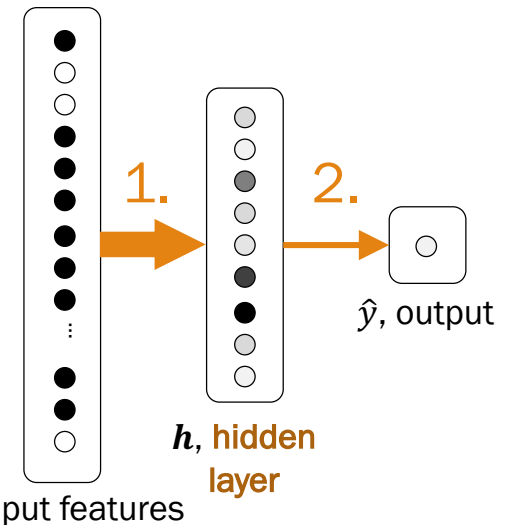
$$h_j = \theta_j^{(h)^T} \mathbf{x}$$

2. $\hat{y} = \sigma(\theta^{(\hat{y})^T} \mathbf{h}) = P(Y = 1 | \mathbf{X} = \mathbf{x})$

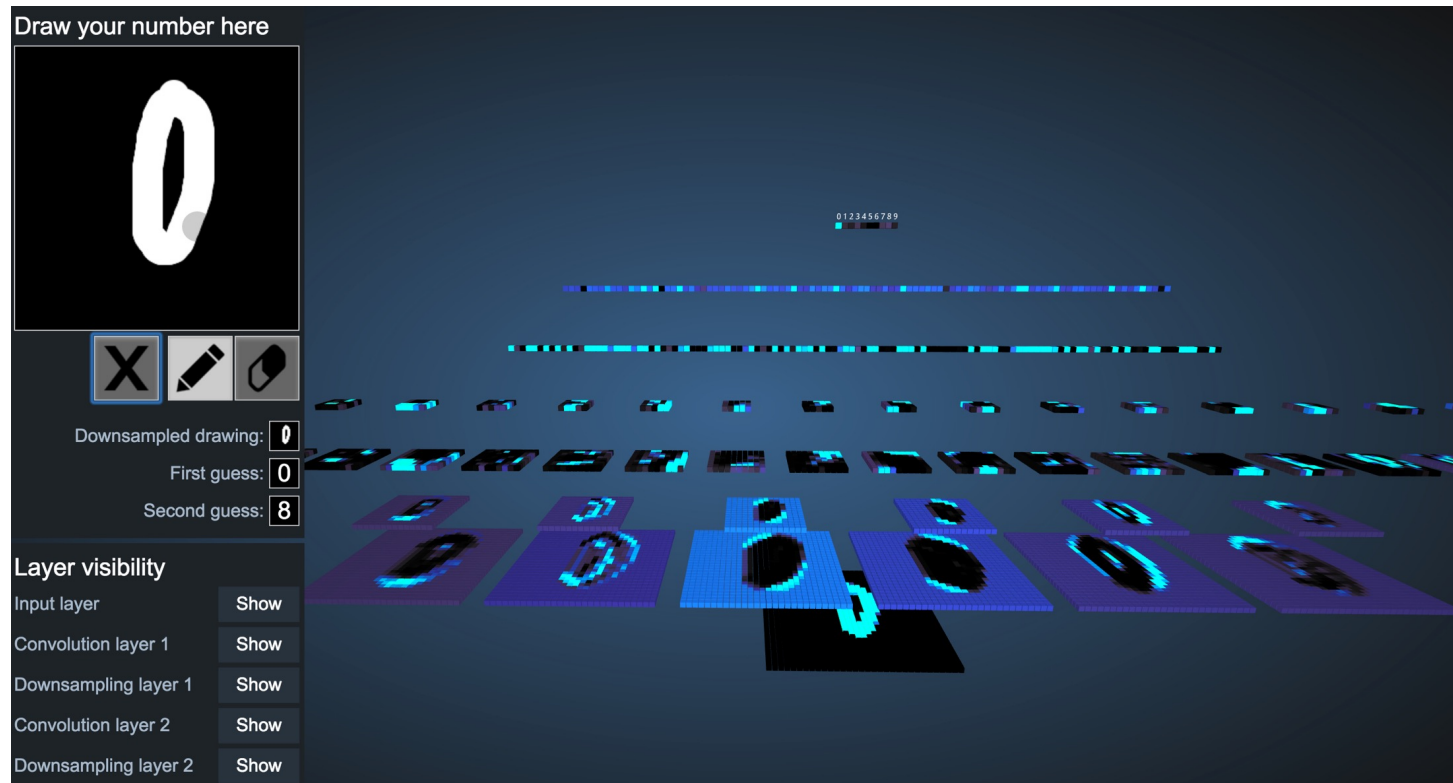
$$[\theta_1 \ \theta_2] \begin{bmatrix} \theta_{11} & \theta_{12} \\ \theta_{21} & \theta_{22} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \Rightarrow \text{linear combinations of}$$

linear combinations of inputs are just linear combinations of inputs.

The linear model is effectively a single logistic regression with $|\mathbf{x}|$ parameters.



Neural Networks and Numeric Digit Recognition



https://adamharley.com/nn_vis/

Neural networks

A neural network (like logistic regression) gets intelligence from its parameters θ .

Training

- Learn parameters θ
 - Find θ_{MLE} that maximizes likelihood of training data (MLE)
- but now, there are many, many layers with many, many parameters.*

Testing/ Prediction

- For input feature vector $\mathbf{X} = \mathbf{x}$:
- Use parameters to compute $\hat{y} = P(Y = 1 | \mathbf{X} = \mathbf{x})$
 - Classify instance as:
$$\begin{cases} 1 & \hat{y} > 0.5 \\ 0 & \text{otherwise} \end{cases}$$
- fundamentally the same as with a single logistic regression.*

Neural networks

A neural network (like logistic regression) gets intelligence from its parameters θ .

Training

- Learn parameters θ
- Find θ_{MLE} that maximizes likelihood of training data (MLE)

How do we learn the $|x| \cdot |h| + |h|$ parameters?

Gradient ascent + chain rule!

Training: Logistic Regression

Review

1. Optimization problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$



$$\hat{y} = \sigma(\theta^T \mathbf{x}^{(i)}) = P(Y = 1 | \mathbf{X} = \mathbf{x})$$

2. Compute gradient

Find $|\mathbf{x}|$ parameters

3. Optimize

```
initialize params
repeat many times:
  compute gradient
  params += η * gradient
```

Training: Neural networks

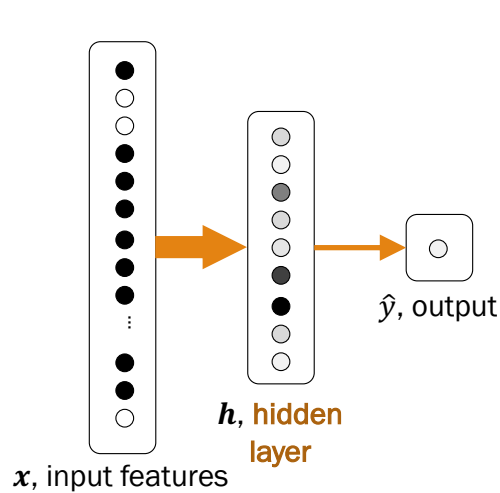
1. Optimization problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

2. Compute gradient

3. Optimize

1. Same output $\hat{y}$, same log conditional likelihood



for $j = 1, \dots, |h|$:

$$h_j = \sigma(\theta_j^{(h)T} \mathbf{x})$$

weights needed to transform $\mathbf{x}$ into h

$$\hat{y} = \sigma(\theta^{(\hat{y})T} \mathbf{h}) = P(Y = 1 | \mathbf{X} = \mathbf{x})$$

weights needed to transform h into $\hat{y}$

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

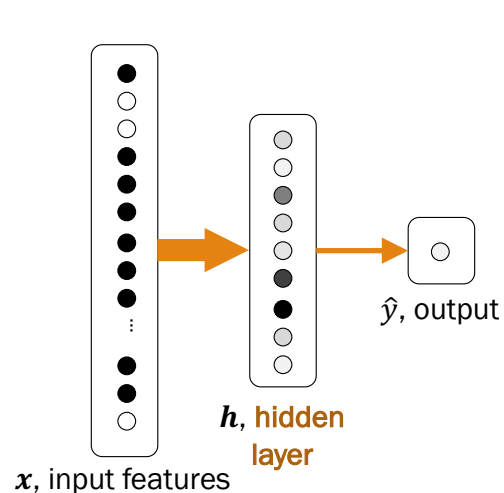
$$L(\theta) = \prod_{i=1}^n P(Y = y^{(i)} | \mathbf{X} = \mathbf{x}^{(i)}, \theta)$$

Binary class labels:
 $Y \in \{0, 1\}$

$$= \prod_{i=1}^n (\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}}$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

(model is a little more complicated)



$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$
$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

for $j = 1, \dots, |\mathbf{h}|$:

$$h_j = \sigma \left(\theta_j^{(h)} \mathbf{x} \right) \quad \text{dimension } |\mathbf{x}|$$

$$\hat{y} = \sigma \left(\theta^{(\hat{y})} \mathbf{h} \right) = P(Y = 1 | \mathbf{X} = \mathbf{x}) \quad \text{dimension } |\mathbf{h}|$$

To optimize for
log conditional likelihood,
we now need to find:

$$|\mathbf{h}| \cdot |\mathbf{x}| + |\mathbf{h}| \quad \text{parameters}$$

2. Compute gradient

1. Optimization problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

$$h_j = \sigma \left(\theta_j^{(h)T} \mathbf{x} \right) \quad \text{for } j = 1, \dots, |\mathbf{h}| \quad \hat{y} = \sigma \left(\theta^{(\hat{y})T} \mathbf{h} \right)$$

2. Compute gradient

Take gradient with respect to all θ parameters

3. Optimize

Calculus refresher #1:

Derivative(sum) =
sum(derivative)

Calculus refresher #2:

Chain rule 🌟🌟🌟

3. Optimize

1. Optimization problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

$$h_j = \sigma \left(\theta_j^{(h)T} \mathbf{x} \right) \quad \text{for } j = 1, \dots, |\mathbf{h}| \quad \hat{y} = \sigma \left(\theta^{(\hat{y})T} \mathbf{h} \right)$$

2. Compute gradient

Take gradient with respect to all θ parameters

3. Optimize

```
initialize params
repeat many times:
    compute gradient
    params +=  $\eta$  * gradient
```

Training a neural net

1. Optimization problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

Wait, did we just skip something difficult?

2. Compute

3. Optimize

initialize params
repeat many times:
 compute gradient
 params += η * gradient

2. Compute gradient via backpropagation

1. Optimization problem:

$$\theta_{MLE} = \arg \max_{\theta} \prod_{i=1}^n f(y^{(i)} | \mathbf{x}^{(i)}, \theta) = \arg \max_{\theta} LL(\theta)$$

$$LL(\theta) = \sum_{i=1}^n y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

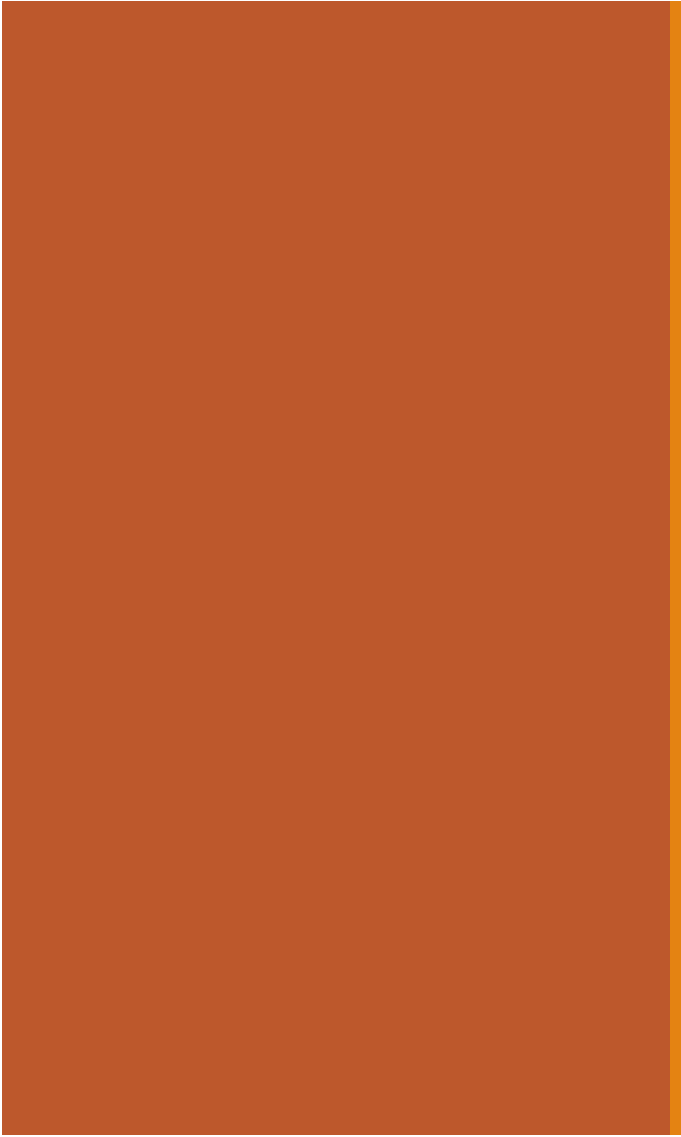
$$h_j = \sigma \left(\boxed{\theta_j^{(h)}}^T \mathbf{x} \right) \quad \text{for } j = 1, \dots, |\mathbf{h}| \quad \hat{y} = \sigma \left(\boxed{\theta^{(\hat{y})}}^T \mathbf{h} \right)$$

2. Compute gradient

Take gradient with respect to all θ parameters

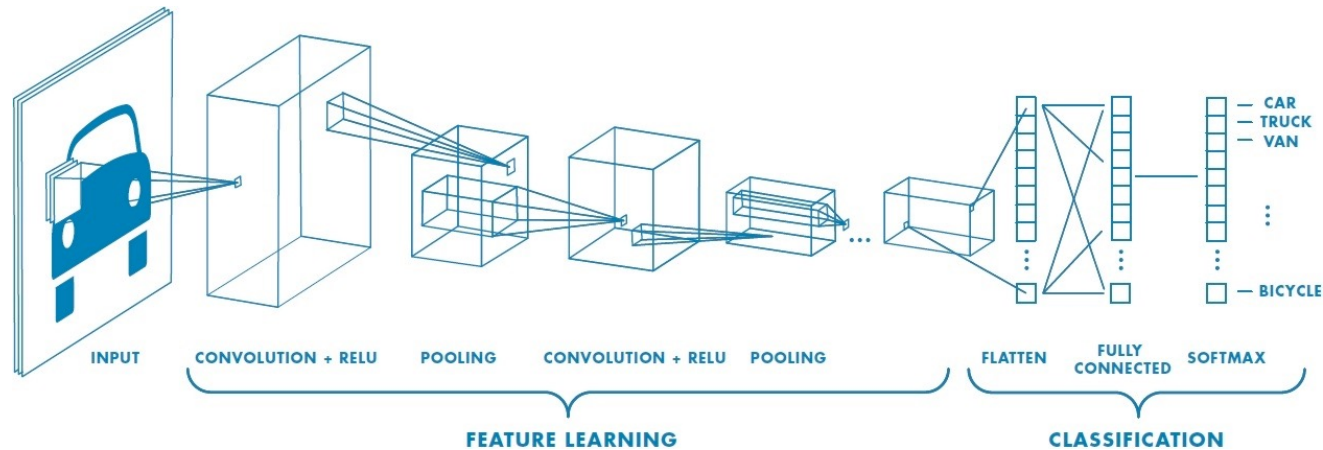
3. Optimize

Learn the tricks behind
backpropagation in
CS229, CS231N, CS224N,
etc.



Beyond the basics

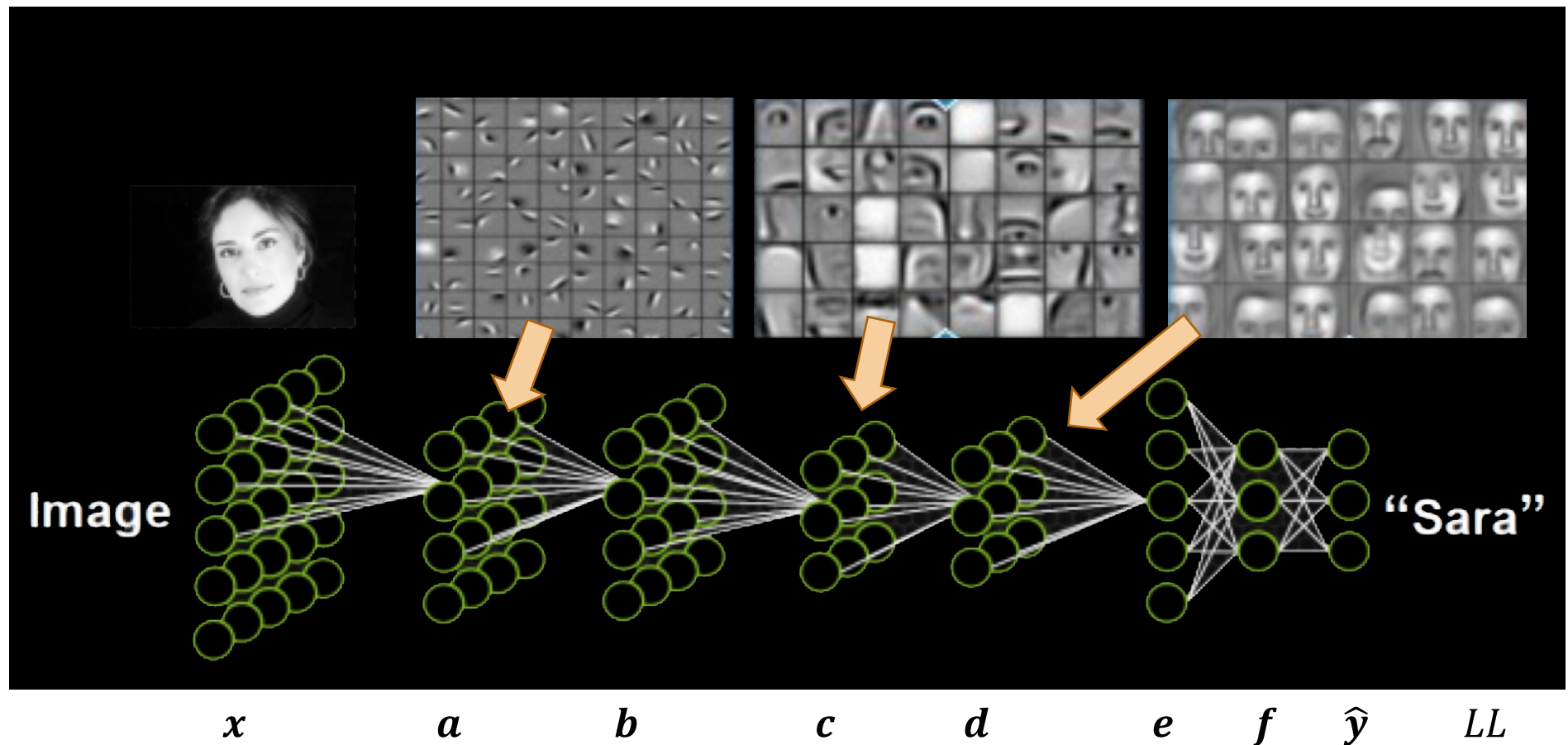
Shared weights?



It turns out if you want to force some of your weights to be shared over different neurons, the math isn't much harder.

Convolution is an example of such weight-sharing and is used a lot in computer vision (Convolutional Neural Networks, CNN).

Neural networks with multiple layers



Lisa Yan, Chris Piech, Mehran Sahami, and Jerry Cain, CS109, Spring 2024

Stanford University

Neurons learn features of the dataset



Neurons in later layers will respond strongly to high-level features of your **training data**.

If your training data is faces, you will get lots of face neurons.

If your training data is all of YouTube...



...you get a cat neuron.



Top stimuli in test set

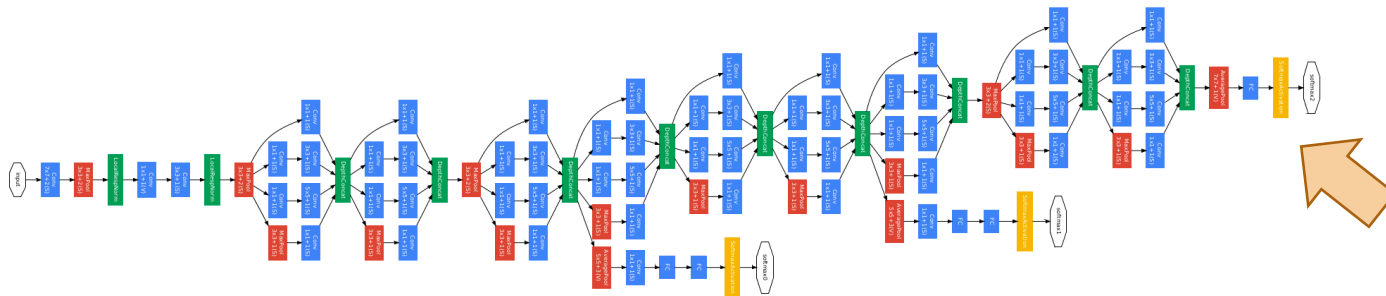


Optimal stimulus found by numerical optimization

GoogLeNet (2015)



1 Trillion Artificial Neurons
(btw human brains have 1 billion neurons)



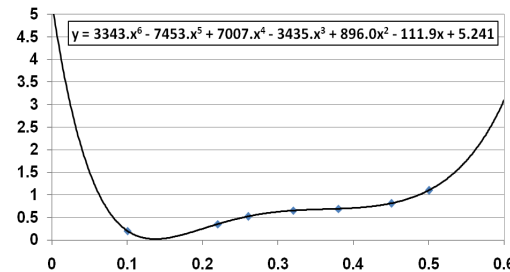
Multiple,
Multi class output

22 layers deep!

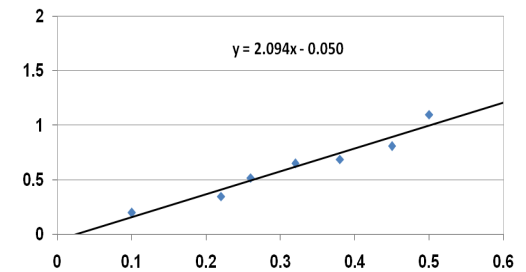
Good ML = Generalization

Overfitting

Fitting the training data too well, such that we lose generality of model for predicting new data



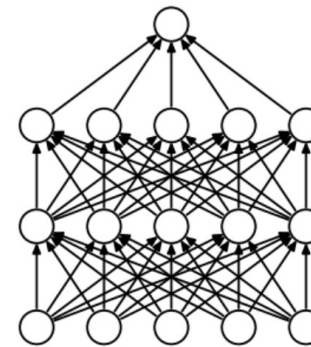
perfect fit, but bad predictor for new data



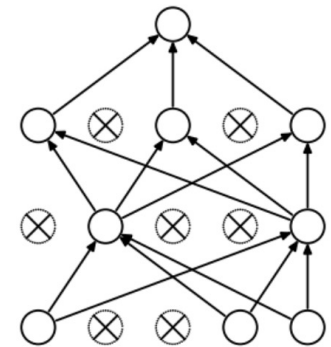
more general fit + better predictor for new data

Dropout

During training, randomly leave out some neurons each training step. It will make your network more robust.



(a) Standard Neural Net



(b) After applying dropout.

Making decisions?



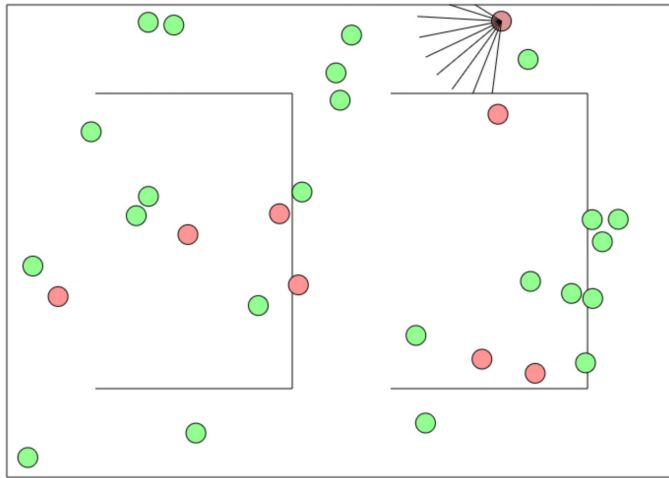
Not everything is classification.

Deep Reinforcement Learning

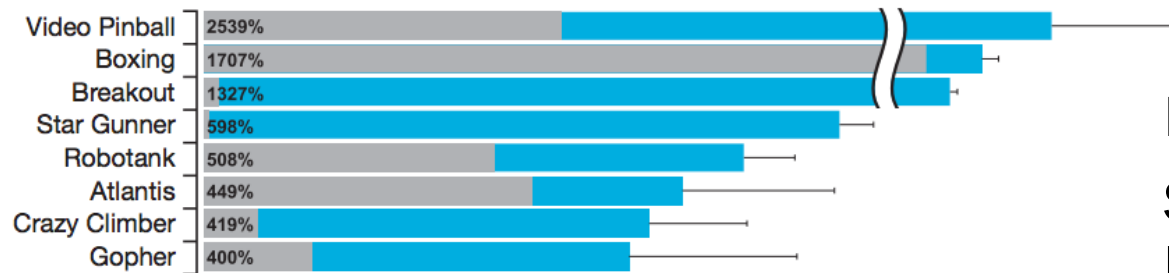
Instead of having the output of a model be a probability, you make output an expectation.

<http://cs.stanford.edu/people/karpathy/convnetjs/demo/rldemo.html>

Deep Reinforcement Learning



<http://cs.stanford.edu/people/karpathy/convnetjs/demo/rldemo.html>



Deep Mind Atari Games
Score compared to best
human